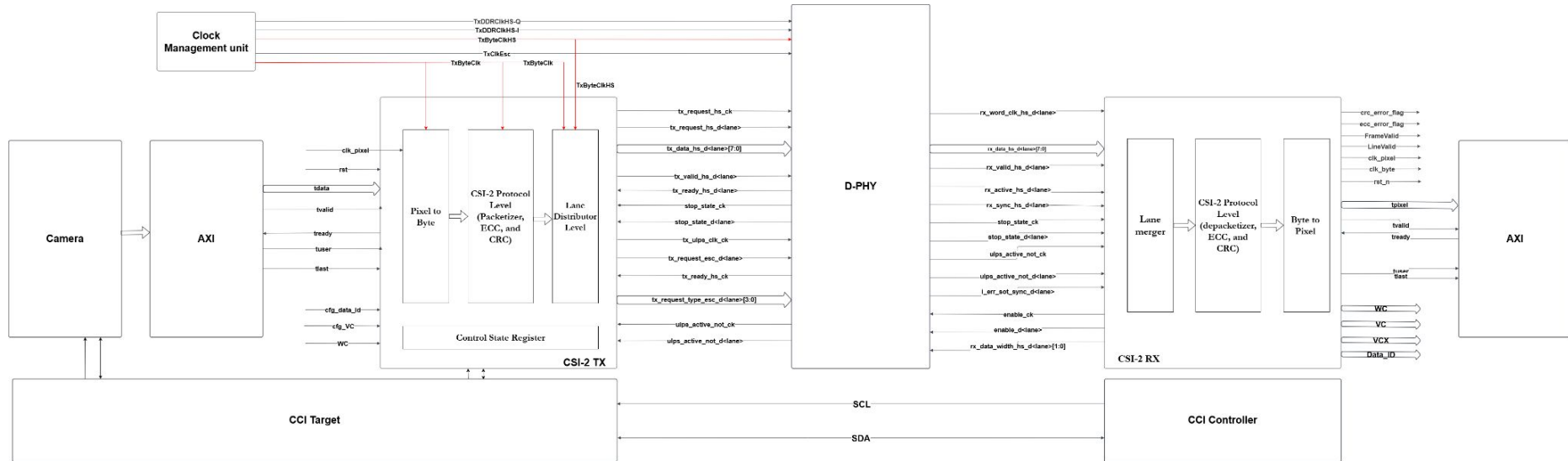
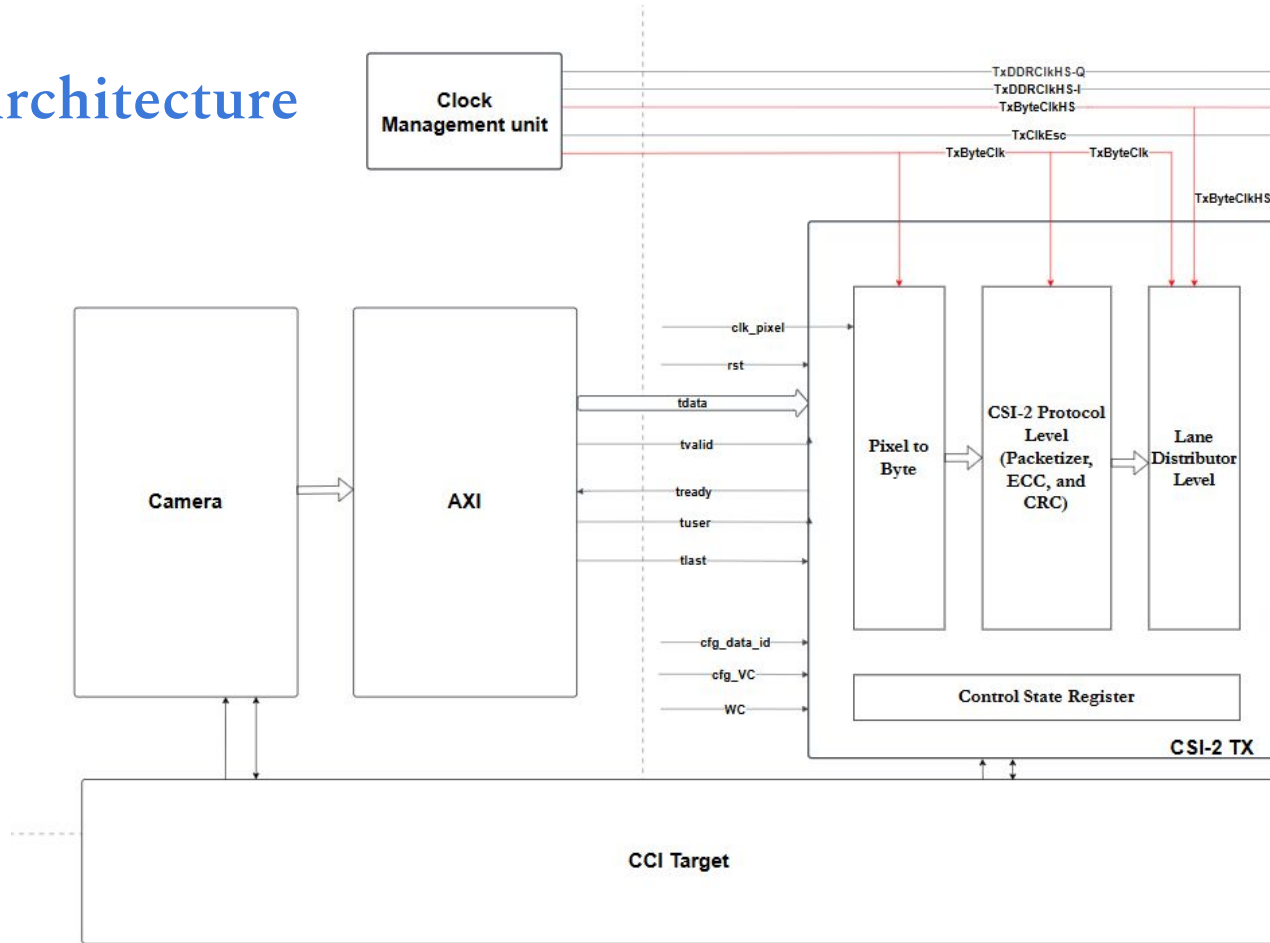


Design and Implementation of CSI Controller IP

System Architecture



System Architecture



Pixel to Byte Converter

Block 1: AXI Input Bridge Logic

Location: The far left box. Clock Domain: Operates on the Pixel Clock .It prepares the data for the FIFO.

Format Mask & Unpacker:

- **The Problem:** The input `axi_tdata` is 32 bits wide, but the pixel might only be 10 bits (RAW10) or 24 bits (RGB888). The extra bits are garbage (padding).
- **The Solution:** Based on your configuration input (`cfg_data_type`), this logic slice the valid bits out of the 32-bit container.
 - *If RAW10:* It grabs `axi_tdata[9:0]`.
 - *If RGB888:* It grabs `axi_tdata[23:0]`.
- **The "Merger":** It takes the "Clean Pixel" and glues the sideband signals (TUSER and TLAST) onto it.
 - *Result:* It creates a single wide data vector (e.g., 26 bits) to send to the FIFO.
 - `fifo_input_data = {axi_tlast, axi_tuser, clean_pixel_data}`

Flow Control:

- It monitors the FIFO's "Full" flag.
- If the FIFO is NOT Full, it sets `axi_tready = 1` (Telling the user "Send more!").
- If the FIFO IS Full, it sets `axi_tready = 0` (Telling the user "Stop!").

Block 2:Asynchronous FIFO

Location: The middle box. **Clock Domain: Dual Clock** (Input = Pixel Clock, Output = Byte Clock). This is the bridge between the slow camera world and the fast MIPI world.

1. Why "Asynchronous"? The camera sends pixels at one speed.MIPI transmitter processes bytes at a totally different speed .The Async FIFO isolates these two timing worlds.

2. The FIFO Structure (Width & Depth):

- **Width:** It must be wide enough to hold the largest pixel **PLUS** the control flags.
 - Max Pixel (RGB888) = 24 bits.
 - Control Flags (TUSER, TLAST) = 2 bits.
 - **Total Width = 26 bits.**
- **Depth :** Standard practice is **1024 or 2048 words**.
 - *Why so deep?* MIPI transmits in bursts. Sometimes it stops to enter "Low Power Mode" (LPS). While the MIPI side is stopped (sleeping), the camera keeps sending pixels. The FIFO must be deep enough to absorb these incoming pixels without overflowing until the MIPI side wakes up.

3.The Signals:

- **wr_en (Write Enable):** Connected to axi_tvalid. Pushes data in.
- **rd_en (Read Enable):** Controlled by the "Gearbox" in the next block. Pops data out.
- **empty:** Tells the next block "I have no pixels, don't try to read."
- **full:** Tells the previous block "I am full, stop sending."

Block 3: Pixel-to-Byte Converter Logic

Location: The right box. Operates on the Byte Clock .

This block takes the "Generic Pixels" and puts them into the specific byte sequence required by the MIPI CSI-2 standard.

1. The Gearbox / Accumulator: This is a shift register that collects pixels until it has enough bits to make full bytes.

- **Logic:** It reads from the FIFO.
 - *Cycle 1:* Read Pixel A (10 bits). Store it. (Current buffer: 10 bits).
 - *Cycle 2:* Read Pixel B (10 bits). Store it. (Current buffer: 20 bits).
 - *Cycle 3:* Read Pixel C (10 bits). Store it. (Current buffer: 30 bits).
 - *Cycle 4:* Read Pixel D (10 bits). (Current buffer: 40 bits).
- **Trigger:** Once the buffer has **40 bits** (for RAW10), it tells the Output Logic: *"ready to convert"*

2. Byte Output Logic: a simple state machine.

- Once the Gearbox is full, this logic pauses the FIFO reading and starts blasting bytes out to the Packetizer.
- *Cycle 1:* Output Byte 0.
- *Cycle 2:* Output Byte 1.
- ...
- *Cycle 5:* Output Byte 4.
- *Done:* It tells the Gearbox "Go read more pixels."

Block 3: Pixel-to-Byte Converter Logic

3. Handling the Sidebands (**sof** / **eo1**):

- When the Gearbox reads a pixel from the FIFO, it also checks bit 24 and 25 (the **TUSER** and **TLAST** we glued on earlier).
- If it sees **TUSER** (Start of Frame), it pauses everything and sends a signal to the **Packetizer** to send a "Frame Start" Short Packet *before* it processes the pixel data.
- From the `cfg_height` , it can calculate the Frame End marker.

CRC Transmitter

CRC Overview

The crc tx (CRC Transmitter) block is a digital logic module designed to ensure data integrity within the CSI-2 transmission protocol.

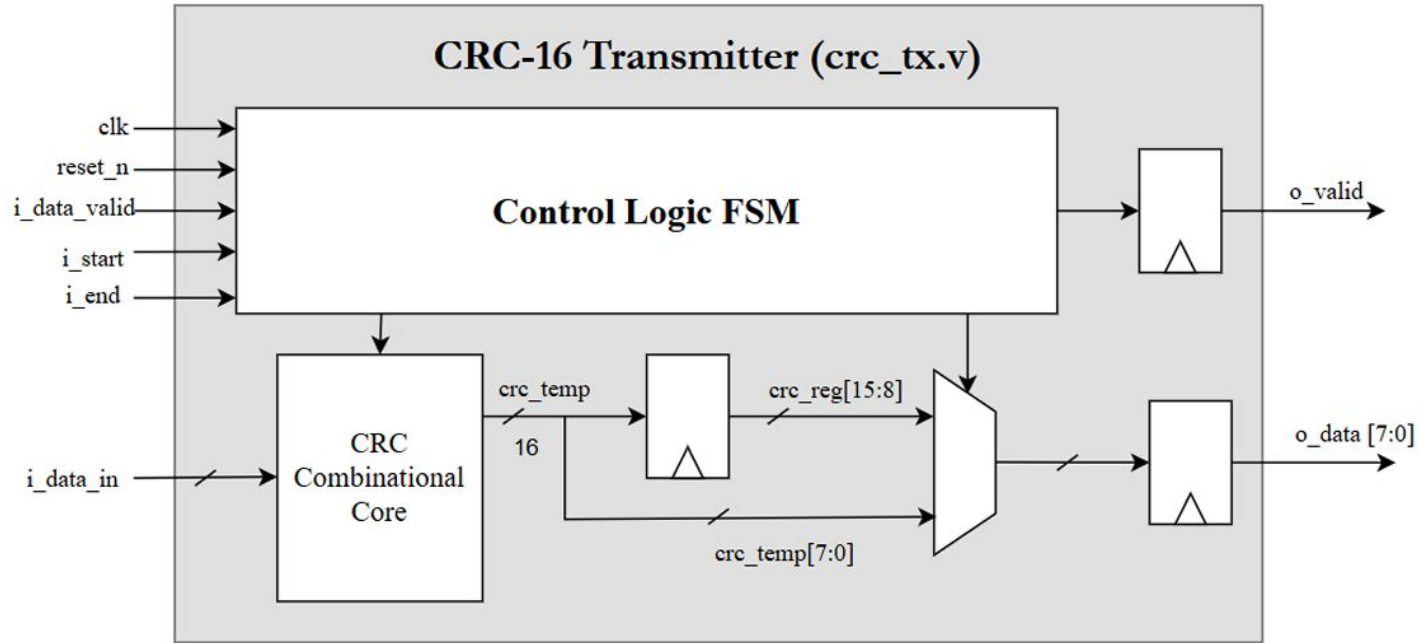
It performs a cyclic redundancy check (CRC) calculation on an incoming stream of 8 bit payload data. The module implements a 16-bit CRC algorithm. Upon the completion of a data packet (signaled by an End-of-Transmission flag), the module appends the calculated 16-bit Checksum to the output stream.

The checksum is transmitted as two sequential bytes: the Least Significant Byte (LSB) followed by the Most Significant Byte (MSB).

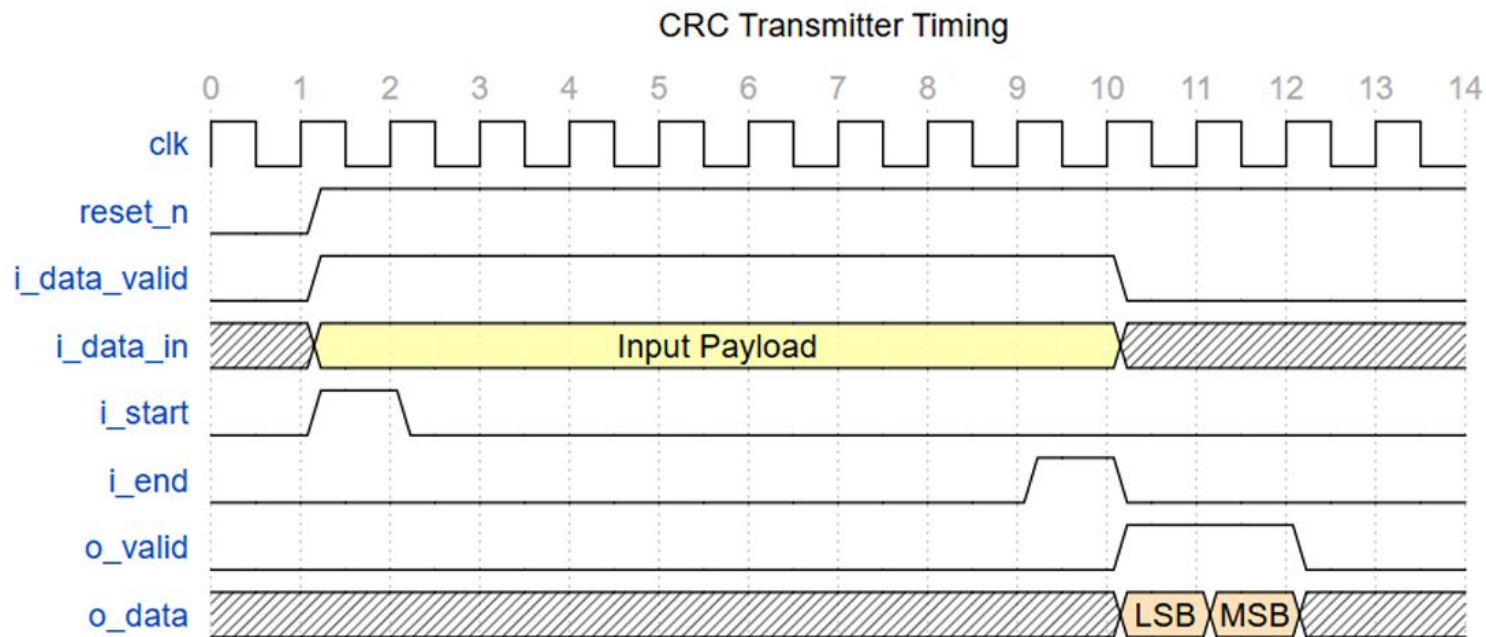
CRC Ports Names

Port Name	Width	Type	Description
clk	1	Input	System Clock. Trigger on rising edge.
reset_n	1	Input	Active-low async reset. Resets CRC to 0xFFFF.
i_data_valid	1	Input	Indicates i_data_in is valid payload.
i_data_in	8	Input	Payload byte input stream.
i_start	1	Input	Start of Transmission (SoT). Resets CRC.
i_end	1	Input	End of Transmission (EoT). Last byte flag.
o_valid	1	Output	Valid high during CRC transmission.
o_data	8	Output	CRC Output bytes (LSB then MSB).

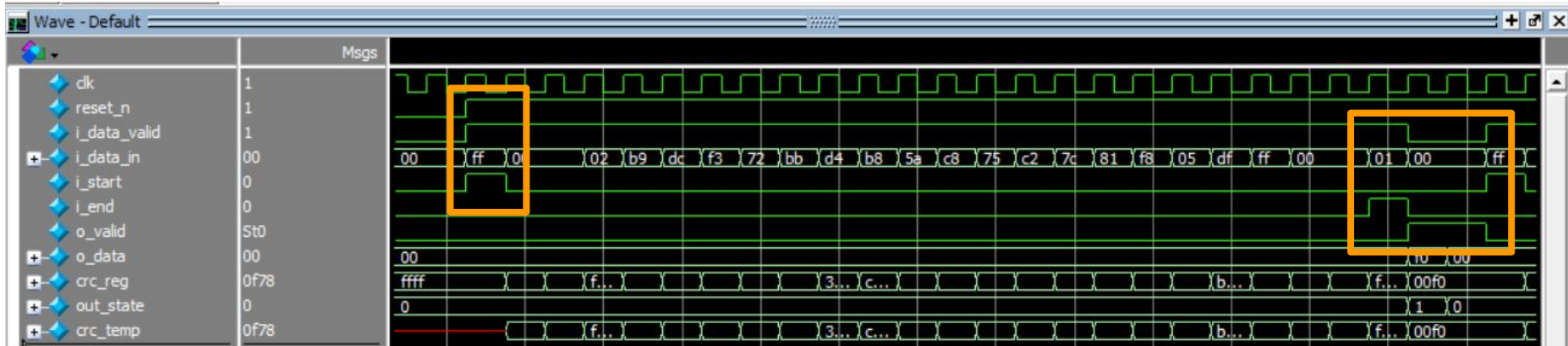
CRC Block Diagram



CRC Timing Diagram



CRC Testbench



```
# IN = 01 (valid=1, start=0, end=1)
# OUT CRC BYTE = f0
# OUT CRC BYTE = 00
# CRC output completed.
#
```

CRC Receiver

CRC Overview

The crc rx (CRC Receiver) block is a digital logic module responsible for verifying data integrity in the MIPI CSI-2 transmission protocol.

The module receives a continuous stream of 8-bit payload data bytes, followed by a 16-bit checksum transmitted as two bytes: Least Significant Byte (LSB) first, then Most Significant Byte (MSB). During payload reception, the crc rx block computes a reflected CRC-16 value in real time.

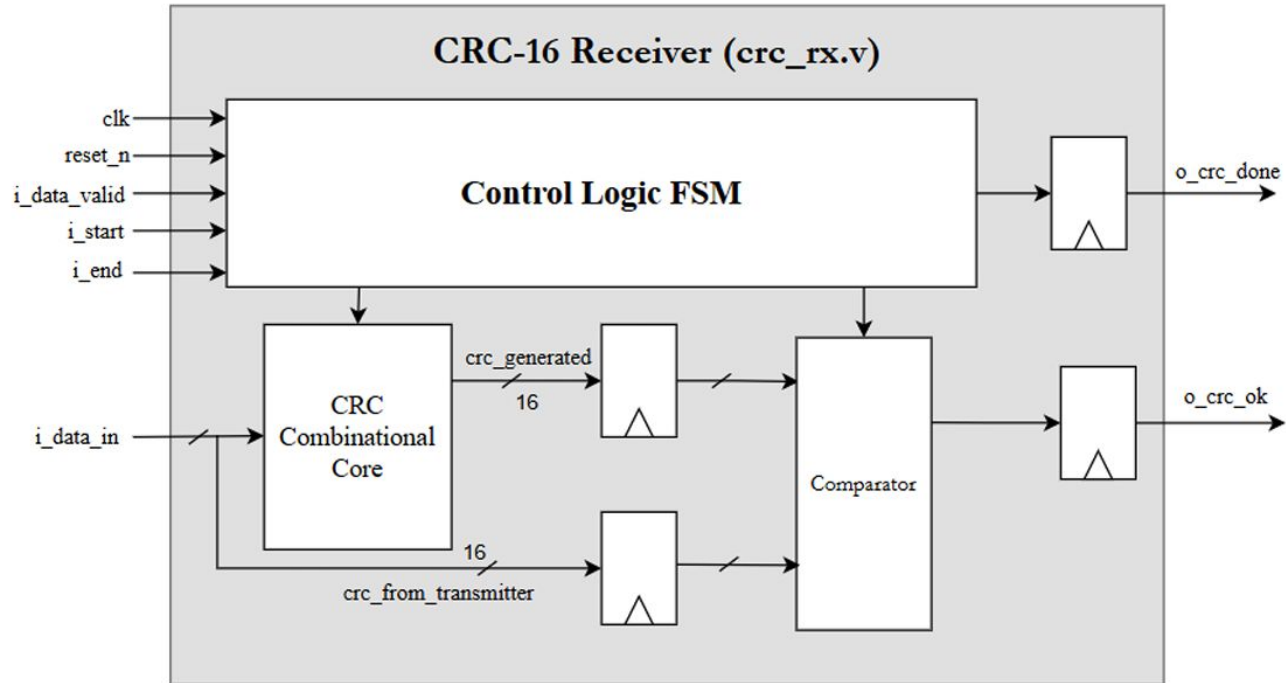
After receiving the final checksum byte (MSB), the internally computed CRC value is immediately compared with the received checksum. Based on this comparison, the module asserts a Pass signal if the packet is valid, or a Fail signal if data corruption is detected.

The CRC validation result is generated with zero additional latency, ensuring immediate packet integrity feedback.

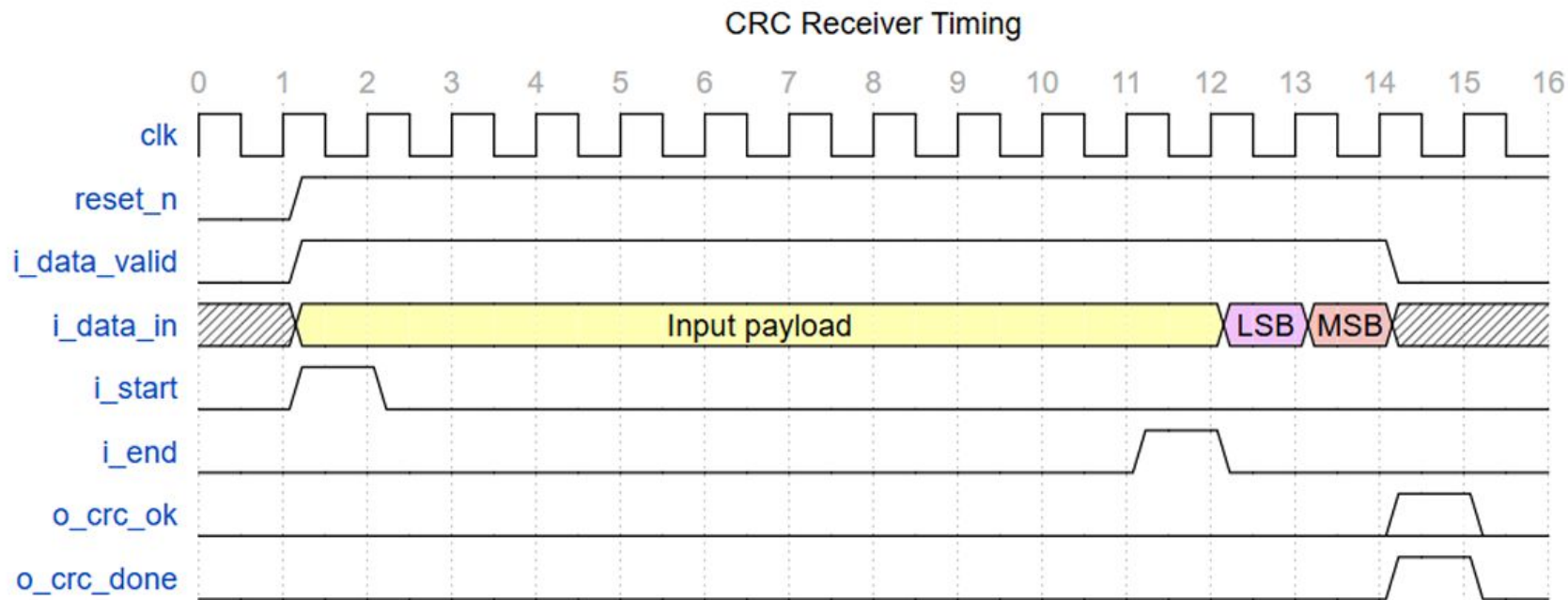
CRC Ports Names

Port Name	Width	Type	Description
clk	1	Input	System Clock. Trigger on rising edge.
reset_n	1	Input	Active-low async reset. Resets CRC to 0xFFFF.
i_data_valid	1	Input	Indicates i_data_in is valid data.
i_data_in	8	Input	Input stream (Payload → LSB → MSB).
i_start	1	Input	Start of Packet (Sync Start with 1st byte).
i_end	1	Input	End of Payload Flag (Marks last data byte).
o_crc_ok	1	Output	High if CRC matches (Pass). Valid when Done is High.
o_crc_done	1	Output	1-cycle pulse indicating check completion.

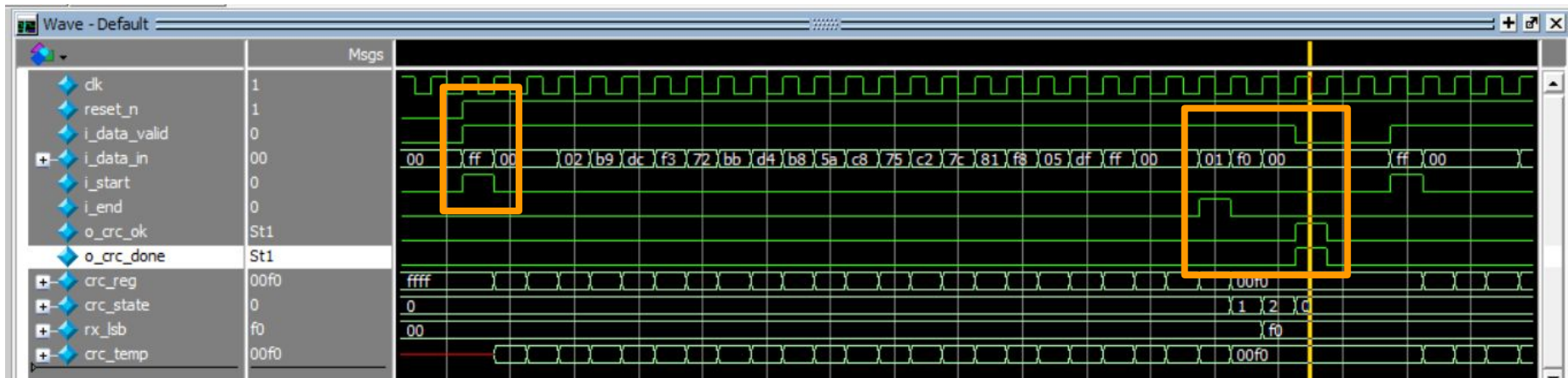
CRC Block Diagram



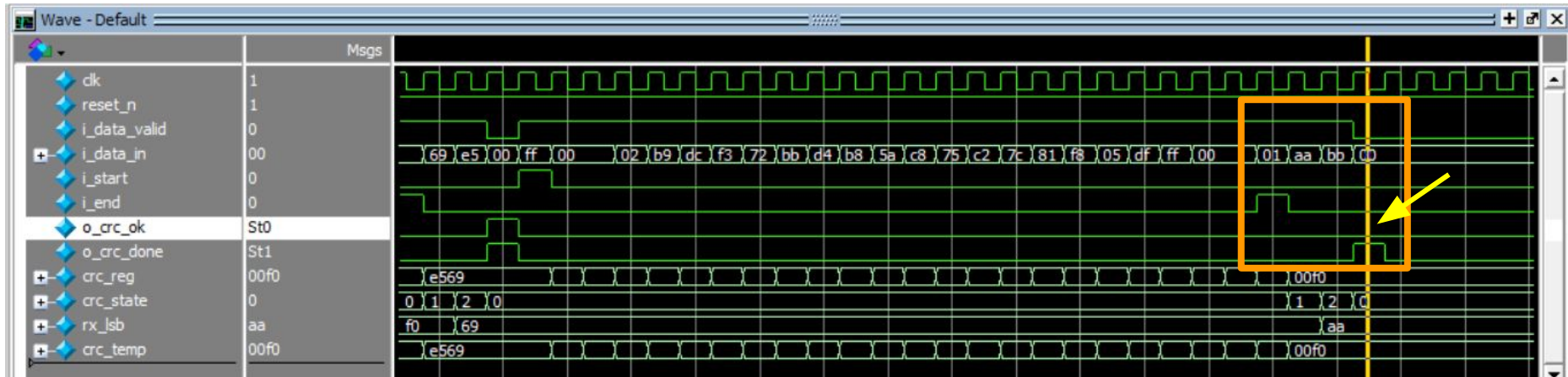
CRC Timing Diagram



CRC Testbench



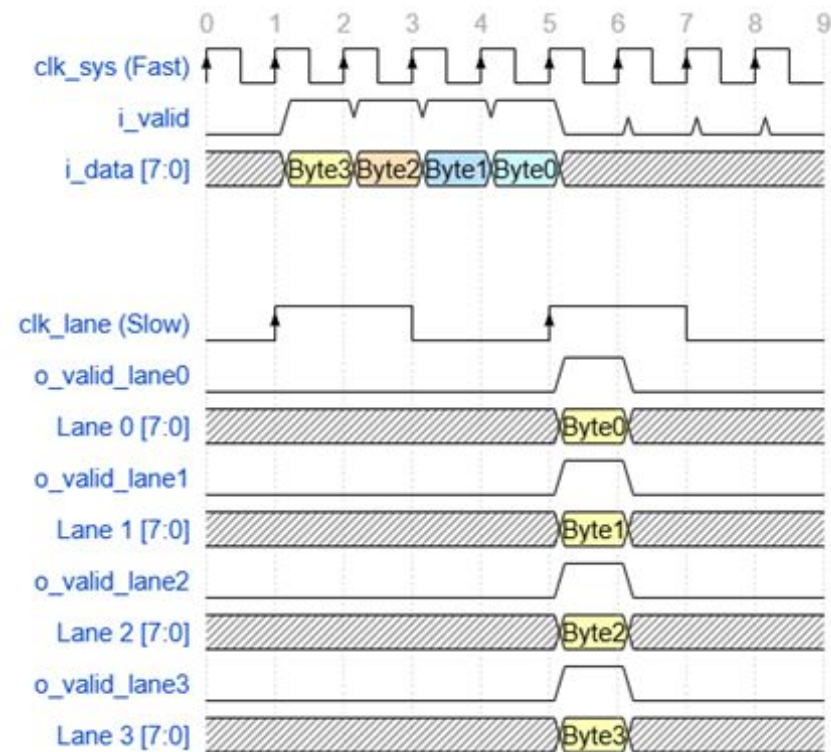
CRC Testbench



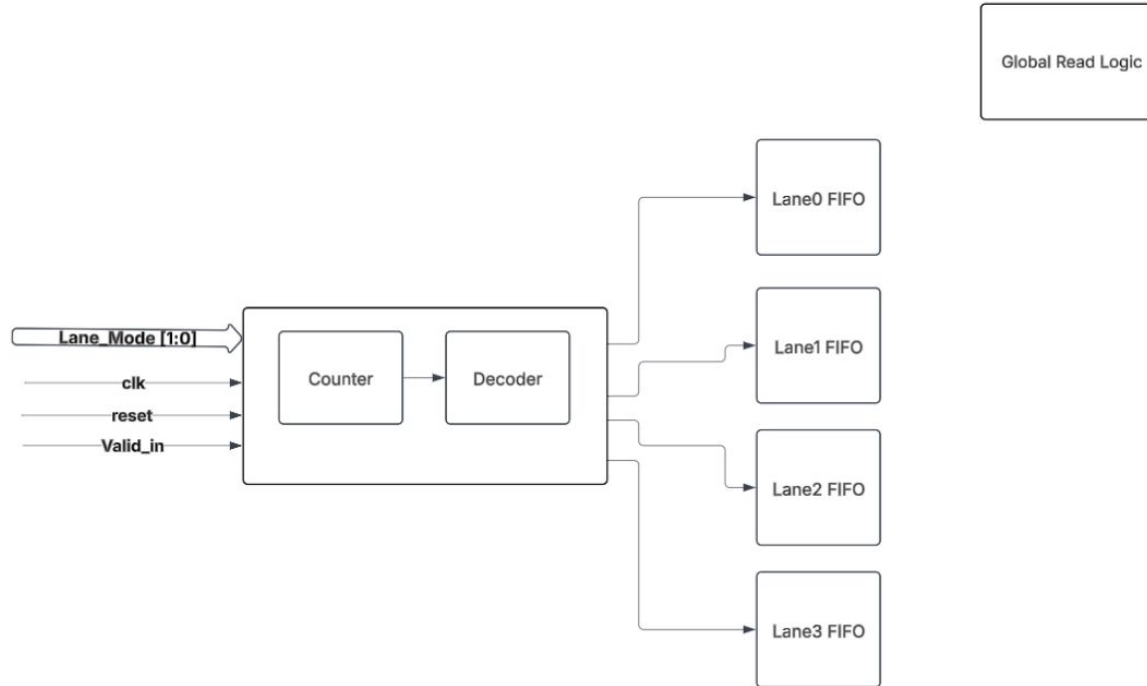
[875000] FAIL: CRC Mismatch Detected (Expected)

Lane Distributor

Lane Distributor



Lane Distributor



References

References

- [1] MIPI Alliance, "Specification for Camera Serial Interface 2 (CSI-2®) Version 4.0.1," MIPI Alliance, May 23, 2022
- [2] MIPI Alliance, "Specification for D-PHY Version 2.5," MIPI Alliance, Jul. 5, 2019. [Online].
- [3] Gowin MIPI Byte to Pixel Converter IP
|Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2753/document/6820dee2cd163.pdf
- [4] Intel Corporation, "*MIPI CSI-2 Intel® FPGA IP User Guide*," Doc. No. 813926, Version 1.1.0, Apr. 26, 2024. Online. Available: <https://www.intel.com/content/www/us/en/docs/programmable/813926.html>
- [5] Gowin MIPI Pixel to Byte Converter IP
|Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2752/document/6820df0ae4095.pdf

Thank You!